

TECHNICAL REPORT

Smart Retail AI

Intelligent Retail Management & Forecasting Platform

Multi-Agent AI Platform with LangGraph, RAG, Forecasting, Azure OpenAI & Azure
Deployment

Capstone Project Report

Left Shift Program 2026 - Data & AI (T5)

Domain: Intelligent Smart Retail System

Prepared for: Left Shift Program 2026 - Data & AI (T5)

Platform: Smart Retail AI

Table of Contents

1. Executive Summary
2. Project Objectives
3. System Architecture
4. Technology Stack
5. Multi-Agent AI System
6. Retrieval-Augmented Generation (RAG)
7. Machine Learning & Forecasting
8. REST API Design
9. Authentication & Security
10. Azure Cloud Integration
11. Deployment & Docker
12. Frontend & Analytics
13. Data Model & Business Workflows
14. Challenges & Optimizations
15. Future Enhancements
16. Conclusion

1. Executive Summary

Smart Retail AI is an end-to-end intelligent retail management platform that connects customers, retailers, warehouse administrators, and AI-powered business intelligence into one full-stack system. Instead of limiting retail automation to a simple storefront, the platform models a real retail supply chain: customers browse and order products, retailers manage inventory and restock requests, administrators control warehouse stock and approvals, and AI agents assist each role with contextual answers, analytics, forecasting, and anomaly detection.

The system combines a FastAPI backend, SQLAlchemy data models, JWT-secured authentication, React.js role-based portals, LangGraph multi-agent orchestration, Pinecone-backed Retrieval-Augmented Generation, Azure OpenAI language models, Prophet forecasting models, anomaly detection tools, Azure Blob Storage for sales backup access, and Docker-based deployment. It exceeds a basic CRUD application by adding role-specific workflows, AI chat assistants, demand forecasting, sales history, low-stock monitoring, restock approval flows, and cloud-ready deployment.

2. Project Objectives

Objective	Description
Role-Based Retail Operations	Provide separate protected portals for customers, retailers, and administrators with workflows tailored to each role.
AI-Assisted Customer Support	Answer customer questions about products, cart status, orders, payments, shipping, return policies, and company information.
Retailer Business Intelligence	Help retailers understand sales, stock, revenue, pending restocks, payments, and top-performing products using an analytics agent.
Demand Forecasting	Forecast product demand and sales trends using trained forecasting models for selected retail products.
Anomaly Detection	Identify abnormal sales activity using model-backed anomaly tools exposed through the retail ML agent.
Cloud Deployment	Package the React frontend and FastAPI backend into a Docker image suitable for Azure App Service or container deployment.
Secure Authentication	Protect APIs with JWT bearer tokens, hashed passwords, and role-aware frontend routing.

3. System Architecture

Smart Retail AI follows a layered architecture designed around real retail roles and AI-assisted decision making:

- Frontend Layer - React 19 application with customer, retail, and admin portals, lazy-loaded routes, Tailwind styling, dashboards, catalog pages, cart, checkout, orders, and chatbots.
- Backend Layer - FastAPI REST application exposing authentication, product, order, retail, warehouse, admin, dashboard, forecast, sales backup, and AI routes.
- Authentication Layer - JWT bearer token flow with password hashing and protected API dependencies.
- Multi-Agent Layer - LangGraph orchestrator routes user and retailer questions to QA, analytics, or retail ML agents.
- RAG Layer - Pinecone vector search over SmartRetailAI policy and company documents, with local text-file fallback.
- ML Layer - Prophet-based product forecasting and anomaly model loading through LangChain tools.
- Database Layer - SQLAlchemy ORM with development SQLite defaults and production-ready database URL configuration.
- Cloud Storage Layer - Azure Blob Storage SAS URL generation for sales backup CSV/text file downloads.
- Deployment Layer - Multi-stage Docker build combining React static assets with the FastAPI runtime.

Workflow Overview

Stage	Description
Step 1	User logs in as customer, retailer, or admin and receives a JWT access token.
Step 2	React ProtectedRoute allows access only to the role's portal.
Step 3	Frontend calls FastAPI endpoints for catalog, cart, orders, dashboard, restock, forecast, or chatbot features.
Step 4	FastAPI validates request data, resolves the current user, and opens a SQLAlchemy database session.
Step 5	AI requests enter /ai/chat for customers or /ai/retail-chat for retailers.
Step 6	LangGraph routes to QA, analytics, or retail ML nodes based on role and query keywords.
Step 7	Agents call RAG retrievers, database tools, forecast models, or anomaly tools.
Step 8	Results return to the frontend as dashboards, chat replies, reports, forecasts, or operational status updates.

4. Technology Stack

Category	Technology	Purpose
Backend Framework	FastAPI, Uvicorn	REST API development and async-ready Python service hosting.
Frontend	React 19, React Router, Tailwind CSS, Framer Motion	Role-based interactive portals and dashboard views.
Database	SQLAlchemy, SQLite/default DB URL, pyodbc	ORM persistence with local development and Azure SQL compatibility.
Authentication	python-jose JWT, passlib bcrypt	Token authentication and secure password hashing.
LLM	AzureChatOpenAI	AI response generation for QA, analytics, and retail ML agents.
Orchestration	LangGraph StateGraph, ToolNode	Routing and tool-using agent workflows.
RAG / Vector DB	Pinecone, LangChain retriever	Semantic retrieval over SmartRetail policy and support documents.
Embeddings	OpenAI / Azure OpenAI embeddings	Vector representation for RAG documents.
Forecasting	Prophet, joblib, pandas	Product-level demand and sales forecast generation.
ML / Anomaly	scikit-learn, xgboost, numpy	Model-backed retail prediction and anomaly detection support.
Cloud Storage	Azure Blob Storage	Sales backup file access through time-limited SAS URLs.
Deployment	Docker multi-stage build	Build React frontend and serve it through the FastAPI container.

5. Multi-Agent AI System

The platform implements a LangGraph-based AI layer that separates customer support, retailer analytics, and machine-learning workflows. This design prevents a single chatbot from handling every concern blindly; instead, the orchestrator directs each question to the agent best suited to answer it.

Agent	Role	Capability
Orchestrator Agent	Router	Inspects user role and query keywords, then routes to QA, analytics, or retail ML.
QA Agent	Customer Support	Uses RAG and database tools to answer policy, cart, order, payment, and product questions.

Agent	Role	Capability
Analytics Agent	Retail Business Analyst	Uses database-backed tools for products, total sales, orders, low stock, top products, pending restocks, and payment summaries.
Retail ML Agent	Forecasting & Anomaly Specialist	Uses LangChain tools for product forecasts and anomaly checks over trained model artifacts.

LangGraph Orchestration

The orchestrator receives query, role, user_id, and database session state. Customer users are routed to the QA agent. Retail users are routed by keyword groups: policy and support questions go to QA, demand and anomaly questions go to the retail ML agent, and sales or inventory questions go to the analytics agent. The graph uses conditional edges from the router node and finishes after the selected specialist node returns a structured response.

6. Retrieval-Augmented Generation (RAG)

The RAG subsystem gives Smart Retail AI a stable knowledge base for company and policy questions. It stores and retrieves text documents covering company information, electronics policy, fashion guidance, home and kitchen essentials, health and personal care, loyalty, refunds, and shipping.

Document Ingestion

- Text documents are loaded from backend/rag/documents.
- RecursiveCharacterTextSplitter creates 500-character chunks with overlap.
- OpenAI or Azure OpenAI embeddings convert chunks into vectors.
- PineconeVectorStore uploads embedded chunks to the configured Pinecone index.

Retrieval

- The QA agent identifies policy and company-information questions through keyword routing.
- retrieve_company_context() queries Pinecone with k=4 relevant chunks.
- If Pinecone is unavailable or not configured, the system falls back to token overlap search across local .txt documents.
- Retrieved context is injected into the system prompt so the LLM answers from known company information instead of unsupported assumptions.

7. Machine Learning & Forecasting

Smart Retail AI includes two complementary ML paths: API-level demand prediction and agent-accessible retail forecasting tools.

Component	Details
Forecast API	GET /ai/forecast returns 7-day and 30-day forecast totals, trend direction, category, and current stock for selected products.
Forecast Models	Prophet model artifacts are loaded with joblib from ml/ml_models using product IDs such as CE100, FA202, HK400, and HP303.
Retail ML Agent Tool	tool_forecast_product maps friendly product names to product IDs and returns predicted sales with a restocking recommendation.
Anomaly Tool	tool_anomaly_check loads anomaly models and evaluates abnormal sales activity using engineered features such as quantity_sold, revenue, rolling_mean_7, lag_1, and lag_7.
DemandForecastService	Loads a demand_forecast_model.pkl and predicts demand from product, category, price, stock, discount, region, month, and day-of-week features.
Stock Interpretation	Prediction outputs are converted into business-friendly labels such as Restocking Required, Low Stock, and Stock Healthy.

8. REST API Design

The backend exposes a broad REST surface across core retail operations and AI capabilities. Major endpoint groups include:

Method	Endpoint	Purpose
POST	/auth/register	Register a user with role and hashed credentials.
POST	/auth/login	Authenticate and issue a JWT access token.
GET	/auth/admins	List administrator users.
GET/POST /PATCH/D ELETE	/products/warehouse	Warehouse product inventory management.
GET/POST /PATCH/D ELETE	/products/retailer	Retailer product catalog management.
POST	/products/approve	Approve or move warehouse products into retailer inventory workflows.
GET/POST /PATCH/D ELETE	/orders/cart	Customer cart management.
POST	/orders/checkout	Create order and payment records from cart contents.
GET	/orders/my-orders	Customer order history.
GET	/orders/retailer-orders	Retailer order management.
PATCH	/orders/{order_id}/status	Update order status during fulfillment.
POST	/ai/chat	Customer chatbot using QA agent.
POST	/ai/retail-chat	Retailer chatbot using orchestrator, analytics, and ML agents.
GET	/ai/forecast	Demand forecast output for featured products.
GET	/retail/restock	Retailer and admin restock request workflows.
GET	/backup/sales-csv	Generate Azure Blob SAS URL for sales backup download.

9. Authentication & Security

Security Pattern	Implementation
JWT Authentication	create_access_token signs user payloads with HS256 and expiry from ACCESS_TOKEN_EXPIRE_MINUTES.
Password Security	passlib bcrypt hashes passwords and verifies login credentials without storing plaintext.
Protected Dependencies	Routes use get_current_user and get_db dependencies to enforce identity and provide scoped database access.
Role-Based UI	React ProtectedRoute limits customer, retail, and admin pages to allowedRoles.
Request Validation	Pydantic schemas validate API payloads, and FastAPI returns validation errors for malformed requests.
Secret Management	Database URL, JWT secret, Azure OpenAI, Pinecone, and Azure Blob settings are loaded from environment variables.
SAS URLs	Azure Blob sales backup access is granted through time-limited SAS tokens rather than public containers.
CORS Control	CORS middleware allows local frontend origins during development while keeping API behavior explicit.

10. Azure Cloud Integration

Azure Service	How Used
Azure OpenAI	AzureChatOpenAI powers QA, analytics, and retail ML agents; Azure/OpenAI embeddings support RAG depending on configuration.
Azure Blob Storage	Stores the sales backup blob and provides secure download links through generate_blob_sas.
Azure SQL Compatibility	The backend uses DATABASE_URL and pyodbc support so the SQLAlchemy layer can move beyond local SQLite.
Azure App Service / Container App	Docker image exposes FastAPI on the platform-provided PORT and serves the built React frontend.

11. Deployment & Docker

The project uses a production-style multi-stage Dockerfile:

- Stage 1 uses node:20-bookworm-slim to install frontend dependencies and build the React app.
- Stage 2 uses python:3.11-slim-bookworm for the FastAPI runtime.
- System packages include build tools, curl, unixODBC, and Microsoft ODBC Driver 18 for SQL Server connectivity.
- Python dependencies are installed from backend/requirements.txt.
- The built frontend is copied into /app/frontend/build and served by FastAPI when available.
- The container starts uvicorn app.main:app on 0.0.0.0 using the PORT environment variable.

This allows one deployed container to host both the API and the compiled frontend, simplifying capstone demonstration and Azure deployment.

12. Frontend & Analytics

The React frontend is organized into role-specific portals with protected route boundaries:

Portal	Routes / Pages	Capabilities
Customer	/user, cart, checkout, orders, chatbot	Browse products, manage cart, complete checkout, view orders, and ask AI support questions.
Retail	/retail, products, forecast, restock, orders, history, chatbot	Manage retailer inventory, inspect demand forecasts, create restock requests, review orders and sales history, and use the retail AI assistant.
Admin	/admin, inventory, approvals, dispatch, logs, users	Manage warehouse stock, approve requests, dispatch inventory, inspect logs, and manage platform users.

Analytics is delivered through both dashboards and the Analytics Agent. Retailers can ask natural-language questions about inventory, revenue, low stock, pending restocks, orders, payments, and top products. The agent converts those questions into database-backed tool calls and returns concise business summaries.

13. Data Model & Business Workflows

The SQLAlchemy data model represents the main retail entities required for a marketplace and supply-chain workflow:

Entity	Purpose
User	Stores email, password hash, role, status, and creation timestamp for customer, retail, and admin identities.
WarehouseProduct	Master warehouse inventory with name, price, quantity, image, and category.
RetailerProduct	Products owned by a retailer and exposed to customers for purchase.

Entity	Purpose
CartItem	Customer cart entries tied to retailer products.
Order / OrderItem	Completed checkout records with retailer, customer, item quantity, and purchase price.
Payment	Order-level payment record with amount, status, method, and timestamp.
RestockRequest	Retailer-to-admin inventory replenishment request with status and notes.
Message	Retailer/admin communication messages with read status.
Sale	ML-ready sales records including revenue, profit, stock, discounts, rolling averages, supplier, warehouse, and region.
ForecastPrediction / AnomalyAlert	Stores prediction and anomaly alert outputs for forecasting workflows.

14. Challenges & Optimizations

Challenge	Solution
Multi-role Complexity	Separated customer, retail, and admin pages into dedicated portal folders and enforced roles through ProtectedRoute.
AI Routing Reliability	Used explicit role and keyword routing in the orchestrator to route predictable retail queries to the right agent.
RAG Availability	Implemented Pinecone retrieval with local document fallback so policy answers still work when cloud vector search is unavailable.
Tool-Grounded Analytics	Analytics answers are backed by SQLAlchemy queries instead of generic LLM guesses.
Forecast Model Access	Mapped friendly product names to model IDs and wrapped model calls in LangChain tools for the retail ML agent.
Deployment Simplicity	Bundled React build output into the FastAPI container, allowing one deployable artifact.
Database Portability	DATABASE_URL configuration and pyodbc support allow development on SQLite and migration toward Azure SQL.

15. Future Enhancements

- Add streamed chatbot responses with Server-Sent Events or WebSockets for a smoother AI experience.
- Persist conversation history per user so agents can maintain context across sessions.
- Expand forecasting coverage beyond the current featured products and store model metadata in the database.
- Move forecast and anomaly model paths into environment configuration for cleaner container deployment.
- Add automated tests for route permissions, checkout behavior, restock transitions, and AI routing decisions.
- Introduce background jobs for scheduled forecast refreshes and anomaly scans.
- Add richer admin audit logs for inventory changes, approvals, dispatches, and user management.
- Use Azure AI Search or a managed Pinecone deployment strategy for scalable production RAG.
- Add role-specific notification delivery for restock approvals, order status changes, and low-stock alerts.

16. Conclusion

Smart Retail AI successfully demonstrates how modern AI engineering can be embedded into a practical retail management platform. The project combines transactional commerce workflows, role-based dashboards, protected APIs, machine-learning forecasts, anomaly detection, RAG-powered support, LangGraph orchestration, and cloud-ready deployment into a cohesive full-stack application.

The platform goes beyond a traditional retail CRUD system by giving customers AI-assisted support, giving retailers business analytics and forecasting intelligence, and giving administrators operational control over warehouse inventory, approvals, dispatch, logs, and user management. Its architecture is modular enough to grow into a production retail assistant with deeper forecasting, richer analytics, and stronger cloud automation.

Prepared for: Left Shift Program 2026 - Data & AI (T5) | Domain: Intelligent Smart Retail System | Platform: Smart Retail AI